Lập trình mạng · Chương 1

Stream: byte stream, contract của read/write, decorator và buffer

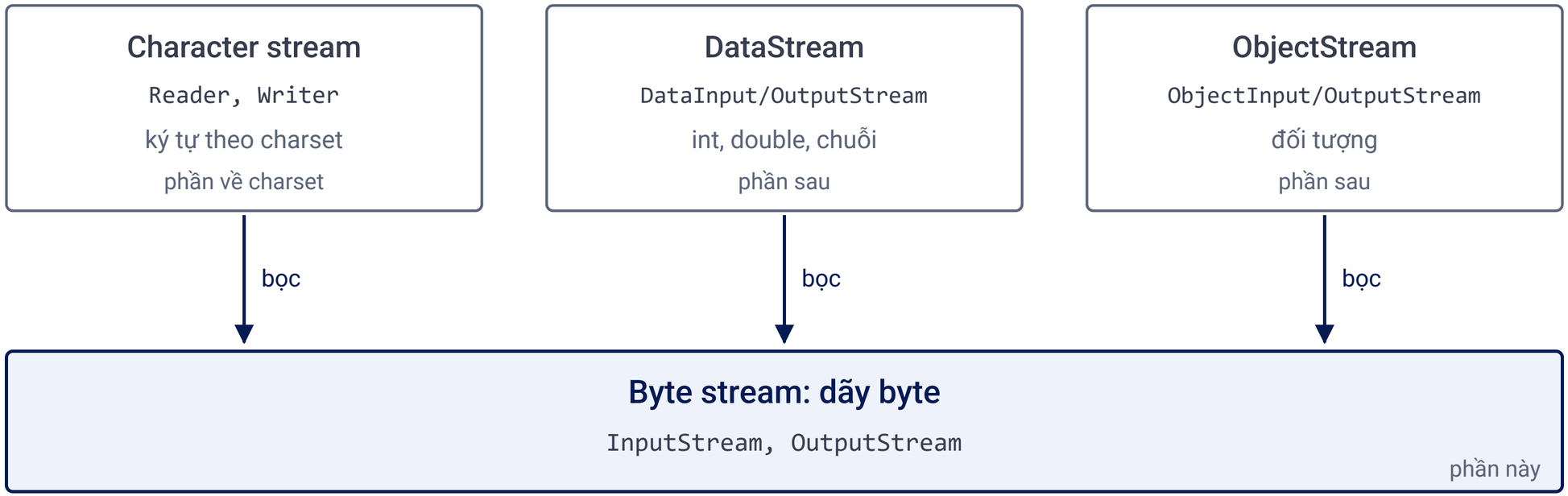
TS. Đặng Ngọc Hùng

Đọc, ghi byte stream sai cách gây thiếu dữ liệu, lặp dữ liệu hoặc chờ không kết thúc

1. Dữ liệu qua mạng đến chương trình Java dưới dạng **byte stream**: dãy byte đến dần theo thời gian
2. Đọc sai contract của `read()`, hoặc ghi xong mà dữ liệu chưa được gửi: **thiếu dữ liệu**, **lặp dữ liệu** hoặc **chờ không kết thúc**

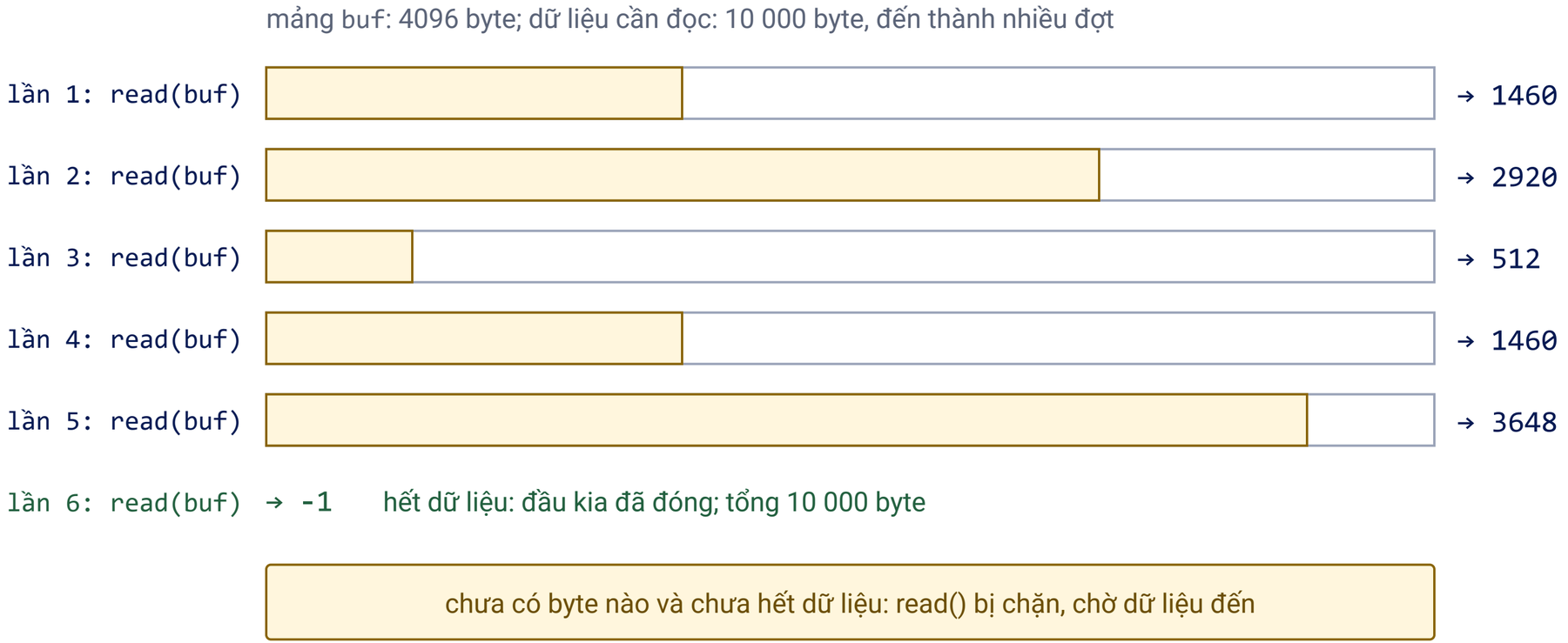
> **Câu hỏi của phần:** đọc và ghi byte stream thế nào cho đúng?

Dữ liệu qua mạng luôn đi qua byte stream, các họ stream khác bọc lên trên



> Dữ liệu qua mạng luôn là **byte**: các họ stream khác đều **bọc byte stream**.

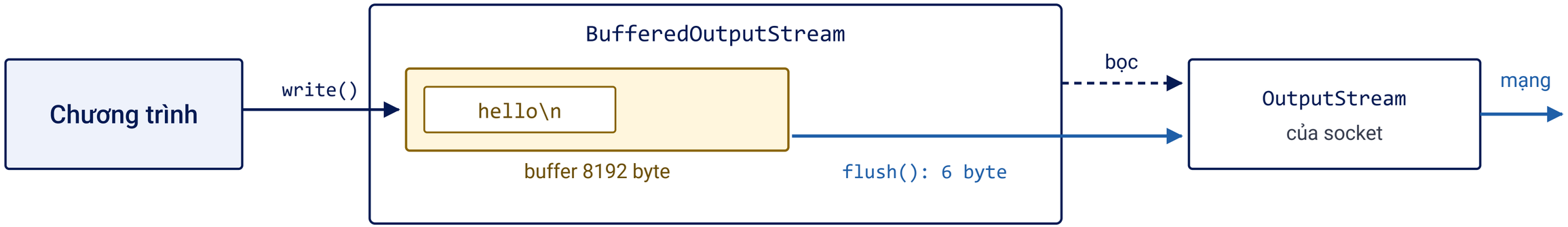
Mỗi lần gọi, read trả từ 1 đến độ dài mảng, hoặc trừ một khi hết dữ liệu



`read(buf)` trả **1 đến buf.length** byte, hoặc **−1** khi hết dữ liệu. Số byte đọc được là **giá trị trả về**.

Muốn đọc đủ dữ liệu: **gọi read lặp lại** đến khi đủ hoặc gặp −1.

BufferedOutputStream bọc OutputStream và giữ byte trong buffer đến khi flush



6 byte nằm trong buffer; stream bên dưới: 0 byte

`close()` = flush() + đóng stream bên dưới; với socket: đóng kết nối

Decorator: một stream bọc stream khác để thêm chức năng; cách gọi `write()` không đổi.

Byte chỉ xuống stream bên dưới khi buffer đầy hoặc khi gọi `flush()`.

Gọi read một lần chỉ nhận được phần dữ liệu đã đến

Java

```
byte[] buf = new byte[4096];
try (InputStream in = new ChunkedInputStream(10_000)) {  // giả lập: dữ liệu đến thành nhiều đợt
    int n = in.read(buf);  // SAI: chỉ gọi read một lần
    System.out.println("read() -> " + n + " of 10000 bytes");
}
```

Kết quả chạy

```
read() -> 1460 of 10000 bytes
```

- Thiếu **8540** byte: phần còn lại chưa đến lúc gọi `read()`, và chương trình không báo lỗi

Gọi read lặp lại đến khi gặp trừ một thì nhận đủ dữ liệu

Java

```
byte[] buf = new byte[4096];
int total = 0;
try (InputStream in = new ChunkedInputStream(10_000)) {  // giả lập: dữ liệu đến thành nhiều đợt
    int n;
    while ((n = in.read(buf)) != -1) {  // -1: hết dữ liệu
        System.out.println("read() -> " + n);
        total += n;  // dùng n, không dùng buf.length
    }
}
System.out.println("total = " + total);
```

Kết quả chạy

```
read() -> 1460
read() -> 2920
read() -> 512
read() -> 1460
read() -> 3648
total = 10000
```

- Năm lần đọc, năm kích thước khác nhau; tổng đủ **10 000** byte

Chưa flush thì dữ liệu vẫn nằm trong buffer

Java

```
var sink = new CountingOutputStream();  // ở vị trí stream của socket
try (var out = new BufferedOutputStream(sink)) {  // decorator: thêm buffer 8192 byte
    out.write("hello\n".getBytes(StandardCharsets.UTF_8));
    System.out.println("after write(): " + sink.count() + " bytes passed down");
    out.flush();
    System.out.println("after flush(): " + sink.count() + " bytes passed down");
}
```

Kết quả chạy

```
after write(): 0 bytes passed down
after flush(): 6 bytes passed down
```

- Trên kết nối mạng: quên `flush()` thì đầu kia không nhận được, **hai bên cùng bị chặn ở** `read()`
- `close()` có flush nhưng đóng luôn kết nối: gọi `flush()` **trước khi chờ phản hồi**

Câu hỏi 1

Gọi read(buf) với buf dài 1024 byte, kết quả trả về 300. Điều nào đúng?

A. 300 byte đầu của buf là dữ liệu mới; có thể còn dữ liệu đến sau, cần gọi read tiếp.

B. Dữ liệu đã hết; lần gọi sau chắc chắn trả -1.

C. Đã xảy ra lỗi mạng, cần đóng kết nối.

D. buf đã chứa đủ 1024 byte dữ liệu mới, nhưng chỉ 300 byte hợp lệ.

Câu hỏi 1

Gọi read(buf) với buf dài 1024 byte, kết quả trả về 300. Điều nào đúng?

A. 300 byte đầu của buf là dữ liệu mới; có thể còn dữ liệu đến sau, cần gọi read tiếp.

B. Dữ liệu đã hết; lần gọi sau chắc chắn trả -1.

C. Đã xảy ra lỗi mạng, cần đóng kết nối.

D. buf đã chứa đủ 1024 byte dữ liệu mới, nhưng chỉ 300 byte hợp lệ.

read trả số byte đọc được, từ 1 đến độ dài mảng; trả ít hơn độ dài mảng không có nghĩa là hết dữ liệu. Chỉ khi read trả -1 mới là hết dữ liệu. Phần sau vị trí 300 của buf là byte cũ.

Câu hỏi 2

Client ghi một dòng yêu cầu qua BufferedOutputStream rồi gọi read để chờ phản hồi. Server không nhận được gì, hai bên cùng chờ. Nguyên nhân là gì?

A. TCP làm mất dòng yêu cầu và không gửi lại.

B. Server chưa gọi flush.

C. BufferedOutputStream chỉ gửi dữ liệu khi gọi close.

D. Client chưa gọi flush, nên dòng yêu cầu còn nằm trong buffer của client.

Câu hỏi 2

Client ghi một dòng yêu cầu qua BufferedOutputStream rồi gọi read để chờ phản hồi. Server không nhận được gì, hai bên cùng chờ. Nguyên nhân là gì?

A. TCP làm mất dòng yêu cầu và không gửi lại.

B. Server chưa gọi flush.

C. BufferedOutputStream chỉ gửi dữ liệu khi gọi close.

D. Client chưa gọi flush, nên dòng yêu cầu còn nằm trong buffer của client.

Byte ghi qua BufferedOutputStream nằm trong buffer đến khi buffer đầy hoặc gọi flush. Client chưa flush thì server không nhận được gì. TCP gửi lại byte bị mất; close cũng đẩy dữ liệu nhưng đóng luôn kết nối.

Tổng kết

1. Dữ liệu qua mạng là **byte stream**; các họ stream khác đều bọc byte stream

2. Contract của `read`: trả **1 đến buf.length** byte hoặc **−1**; dùng giá trị trả về, gọi lặp đến khi đủ

3. **Decorator** `BufferedOutputStream` thêm buffer: gọi `flush()` trước khi chờ phản hồi

Từ khóa cần nhớ byte stream · InputStream · OutputStream · contract của read · decorator · buffer · flush

Mở rộng

Chủ đề nâng cao

- readNBytes và readAllBytes: khi nào dùng, giới hạn bộ nhớ
- transferTo: sao chép stream không cần vòng lặp

Bài tập mở rộng

Viết hàm readFully(InputStream, int n) đọc đúng n byte hoặc ném EOFException; kiểm thử với stream trả dữ liệu từng phần.

Tiếp theo

Stream: dữ liệu có kiểu với DataStream và ObjectStream